Resumo Executivo — Avanços do Cronograma de Observabilidade

Produto: **AutoChecker** · Ferramenta: Sentry (org `automind-qd`) · backend Django · worker Celery · frontend Next.js

Data da revisão: **15/06/2026** · Base: planilha do cronograma + auditoria código×Sentry + análise ao vivo (MCP)

7 ondas · 31 itens | Foco atual: Onda 0 (gate de saneamento) | Citações verificadas arquivo:linha

Esta revisão pontua **todos os avanços** registrados no cronograma de observabilidade do AutoChecker até 15/06/2026, separando o que já mudou *no produto e no Sentry* do trabalho de *diagnóstico e planejamento* que sustenta a execução. O número é deliberadamente honesto: a maior parte dos 31 itens ainda não começou — o avanço está concentrado na **Onda 0** (o gate de saneamento) e em uma base de diagnóstico hoje completa e auditada.

1. Onde estamos — em números

1 Concluído (Item 1 · environment)	**2** Em andamento (Itens 2 e 6)	**1** Pendente (Item 3 · KeyError Beat)	**27** Não iniciados (Ondas 1–6)

~1.640 eventos do maior ruído histórico neutralizados (Item 3)	**~1.800** eventos de ruído mapeados p/ remover sem deploy (quick wins)	**1** incidente real ativo trava o gate (Item 4 · drift de migração)

Leitura executiva: a Onda 0 está em curso e já reduziu o maior ruído histórico do produto, mas o **gate ainda não foi liberado** — há 1 incidente real ativo (drift de migração multi-tenant) que precisa ser resolvido antes de ampliar para alertas, tracing e métricas. Em paralelo, a base de diagnóstico (cronograma auditado + detalhamento "como resolver" das Ondas 0–3 + revalidação ao vivo) está **pronta**, o que permite executar com baixo risco.

2. Avanços de execução — no produto e no Sentry

Mudanças concretas, verificadas `arquivo:linha` contra o código real e cruzadas com o Sentry ao vivo (15/06):

Item 1 — Padronizar `environment` nos 3 SDKs + isolar DEV/PRD Concluído

Os três SDKs (backend, worker, frontend) passaram a padronizar a tag `environment`. Era a base para tudo: sem ambiente correto, qualquer painel é estatisticamente sem sentido.

Ressalva ao vivo (15/06): ainda persiste o fallback `"development"` quando a env var falta (`base.py:730`, `celery.py:24`, configs do frontend) e os alertas seguem *sem filtro de ambiente* → considerar **parcial** até fechar o lado de deploy e atingir o KPI ≥99% de eventos com ambiente correto.

Item 6 — `beforeSend` / Inbound Filters (filtro de ruído) Em andamento

O `before_send` do backend, antes "morto", está agora **ATIVO**: descarta o caminho de health e blips transientes de Redis (`backend/config/settings/base.py:713-721`). O worker também ganhou um `before_send` parcial (`worker/config/celery.py:29-31`, só health).

Falta: reclassificar o ruído de autenticação JWT por endpoint (sem dropar incidentes reais), o Inbound Filter de `DisallowedHost` (scanners) e os filtros do frontend.

Item 3 — PeriodicTask órfã do Beat (`KeyError process_scheduled_notifications`) Pendente

Era o **maior ofensor histórico** do produto (~1.640 eventos). A task foi **comentada no código** (`notification_tasks.py:49-67`) e a issue está **dormente há 26 dias** no Sentry — saiu da lista de ativos por recência.

Falta para fechar: expurgar a entrada órfã persistida (arquivo/registro do django-celery-beat) e resolver/ignorar a issue no painel (1 clique).

Item 2 — Release padronizado + set-commits + isolar pipelines Azure Em andamento

Mapeado e em curso, mas **sem evidência de conclusão** na revisão ao vivo. Depende de mexer nas 4 pipelines Azure e de eleger o `next.config.js` canônico do frontend (hoje aponta para projeto Sentry errado por default).

Desbloqueia "suspect commits" e a correlação picos × deploys.

3. Avanços de diagnóstico e planejamento — a base que destrava a execução

Boa parte do valor entregue até aqui não é código: é a fundação que transforma o cronograma de "lista de boas intenções" em plano executável, item a item, sem chutes.

- **Cronograma completo e auditado** — 7 ondas, 31 itens, com esforço (P/M/G), camada, semana e evidência por linha. A Onda 0 é um *gate* explícito: nada de ampliação antes de sanear.
- **Auditoria cronograma × código (10/06)** — verificação `arquivo:linha` que corrigiu **8 premissas falsas** do plano original (ex.: o health check é uma view trivial que não checa dependências; o helper de init não dá `CeleryIntegration` ao backend; 17 `except` sem captura em `composition.py`; drift afeta tenants pré-existentes, não a criação).

- **Detalhamento "como resolver cada etapa" — Ondas 0 a 3 (itens 1–21)** — estudo fundamentado em 3 fontes (código real + documentação + Sentry ao vivo), com flags *Sei / Parcial / Não sei* e saída em md + html. É a fonte canônica de execução.
- **Quick wins da Onda 0 (15/06)** — sequência de ações de baixo custo que removem **~1.800 eventos de ruído sem deploy e sem risco** (Inbound Filter de `DisallowedHost`, fechar a KeyError dormente, filtro de ambiente nos 3 alertas).
- **Revalidação Sentry ao vivo (15/06)** — coleta via MCP cruzada com o código, separando RUÍDO de INCIDENTE REAL e registrando o que mudou desde 12/06.

4. O que ainda trava o gate da Onda 0

Item	Pendência que segura o gate	Status	Criticidade
4	Drift de migração multi-tenant (`relation "..." does not exist`) — deploy roda `migrate`, nunca `migrate_schemas`; tenants pré-existentes ficam sem as tabelas novas.	Não iniciado	🔴 Incidente real ativo (nº 1)
5	Consolidar init do Sentry no helper (+ `CeleryIntegration` no backend) — exige alterar o helper, não só trocar a chamada.	Não iniciado	Média
7	Scrubbing de PII/LGPD no frontend — `sendDefaultPii:true` persiste; Session Replay sem masking sobre telas de CNH/ASO/CPF.	Não iniciado	Alta (compliance)
8	Triagem ZERO do backlog — **0 issues resolvidas** em 7 dias nos 3 projetos; backlog cresce sem fechamento.	Não iniciado	Média

5. Próximos passos recomendados (priorizados)

#	Ação	Item	Por quê agora
1	**Backfill** `migrate_schemas` nos tenants em drift + gate `--check` no deploy.	4	Único incidente real ativo (visto há 1–2 h); gera centenas de eventos/dia no worker.
2	Aplicar os **quick wins de painel**: Inbound Filter de `DisallowedHost`, fechar a KeyError dormente, filtro de ambiente nos 3 alertas.	6/3/1	Remove ~1.800 eventos de ruído sem deploy e sem risco; destrava a confiança nos alertas.
3	**Reclassificar ruído de JWT** por endpoint (warning/info para anônimo esperado) + ampliar `JWT_SKIP_PATHS` com rotas públicas reais.	6	Maior fonte de ruído do backend; novas rotas entrando toda semana.
4	**Desligar** `sendDefaultPii` no frontend e investigar o alerta "Potential Security Risk" (`.env`).	7	Risco LGPD + possível exposição de segredo em payload.
5	Fechar os pontos abertos do **Item 1** (env var no deploy, KPI ≥99%) e do **Item 2** (release/set-commits nas pipelines).	1/2	Conclui formalmente o que já está "em andamento" e libera correlação deploy × picos.

Resumo da revisão: a Onda 0 saiu do papel — o maior ruído histórico foi neutralizado, o `before_send` está ativo e a base de diagnóstico está completa e auditada. Para **liberar o gate** e habilitar as ondas de alertas, tracing e métricas, o caminho crítico é resolver o drift de migração (Item 4), aplicar os quick wins de painel e fechar PII/triagem.

Revisão gerada em 15/06/2026. Status extraído de `Cronograma_Observabilidade_AutoChecker_2026-06-03.xlsx`; avanços de execução e ressalvas verificados `arquivo:linha` contra o código real em `Produtos\Autochecker` e cruzados com o Sentry ao vivo (org `automind-qd`, janela 7d) via MCP. Fontes de apoio: detalhamento Ondas 0–3 (15/06), Quick Wins Onda 0 (15/06), análise Sentry × código (15/06).